

CCS112

Application Development and Emerging Technologies

Final Module Handout

*Module 1: Introduction to RESTful APIs • Module 2: Cloud Computing and Deployment
• Module 3: Emerging Technologies (AI)*

Weeks 13-17

Prepared for course use • jdcaragenas@pnc.edu.ph
July 2026

Table of Contents

Table of Contents.....	2
Course Intended Learning Outcomes (CILOs)	3
Module 1: Introduction to RESTful APIs (Weeks 13-14).....	4
CILO-to-ILO Alignment Table.....	4
(a) Understanding API and Design Principles	4
(b) HTTP Fundamentals.....	6
(c) Building a Simple RESTful API.....	7
Laboratory Activities	9
Lab 1: Implement CRUD Endpoints Mapped to HTTP Verbs and Resource-Oriented URIs	9
Lab 2: Develop Validation Middleware and Structured Error Responses.....	11
Lab 3: Construct a Persistence Layer That Reliably Commits State Changes.....	12
Module 2: Cloud Computing and Deployment (Week 15)	14
CILO-to-ILO Alignment Table.....	14
(a) Introduction to Cloud Computing Platforms.....	14
(b) Serverless Architecture and Microservices	15
(c) Deployment Strategies for Scalable Applications.....	17
Laboratory Activities	18
Lab 1: Access and Navigate a Cloud Platform to Deploy and Manage Simple Resources	18
Lab 2: Implement a Basic Serverless Function on a Cloud Platform	19
Lab 3: Deploy a Sample Application Using Cloud Services with Scalable Configurations	20
Module 3: Emerging Technologies: Artificial Intelligence (Weeks 16-17)	22
CILO-to-ILO Alignment Table.....	22
(a) Introduction to AI and Machine Learning	22
(b) AI in Application Development	23
(c) Ethical Considerations in AI Application Development	25
Laboratory Activities	26
Lab 1: Apply Basic Machine Learning Algorithms on Sample Datasets	26
Lab 2: Implement a Simple AI Feature in a Sample Application	27
Lab 3: Analyze Sample AI Models or Datasets for Ethical Concerns.....	29

Course Intended Learning Outcomes (CILOs)

The table below lists the three Course Intended Learning Outcomes (CILOs) referenced throughout this handout. Each Module Intended Learning Outcome (ILO) in Modules 1-3 is mapped to the CILO(s) it supports in the alignment tables that follow. This handout carries forward the same three CILOs used in the midterm module handout for this course.

CILO	Description
CILO 1	Develop specifications for a software development effort that precisely articulates the functional requirements, expected execution paths, and the explicit use of cutting-edge or emerging technologies -- including hardware devices and software library APIs.
CILO 2	Select and apply defined coding, documentation, and licensing standards on a sufficiently complex software project, using coding idioms and design mechanisms to achieve reliability, efficiency, and robustness, with respect to legal and ethical considerations.
CILO 3	Undertake, as part of a team activity, source code inspection and unit testing of the functional units of a sufficiently complex software project.

Module 1: Introduction to RESTful APIs (Weeks 13-14)

Mapped CILOs (as given): CILO 1, CILO 2, CILO 3

Topics: (a) Understanding API and Design Principles, (b) HTTP Fundamentals, (c) Building a Simple RESTful API

CILO-to-ILO Alignment Table

#	Intended Learning Outcome (ILO)	CILO(s) Supported
Lecture ILOs		
1	Deconstruct the core constraints of the REST architectural style and evaluate how they promote scalability, separation of concerns, and statelessness in modern web applications.	CILO 1
2	Explain the lifecycle of an HTTP request/response cycle, distinguishing the precise roles of URI structures, HTTP methods (GET, POST, PUT, DELETE, PATCH), headers, and status codes.	CILO 1
3	Formulate robust strategies for API versioning, error handling (using standardized JSON payloads), and stateless authentication.	CILO 1, CILO 2
Laboratory ILOs		
1	Implement a functional backend service that exposes standard CRUD endpoints mapping directly to appropriate HTTP verbs and resource-oriented URI paths.	CILO 1, CILO 2
2	Develop middleware or controller-level validation logic to intercept incoming requests, sanitize inputs, and return meaningful, structured error messages for malformed data.	CILO 2
3	Construct data persistence layers within the API framework to ensure that state modifications initiated by REST clients are correctly committed to a backend data store.	CILO 1, CILO 2

(a) Understanding API and Design Principles

An Application Programming Interface (API) is a contract that allows two software systems to communicate: one system exposes a set of operations and data structures, and the other consumes them without needing to know the internal implementation details. REST (Representational State Transfer) is an architectural style for designing networked APIs, first articulated by Roy Fielding in his 2000 doctoral dissertation. Rather than being a protocol or a strict standard, REST is a set of constraints that, when applied consistently, produce APIs that are scalable, loosely coupled, and easy to evolve over time.

REST defines six architectural constraints. Client-server separation ensures the user interface and the data storage concerns are independent, allowing each to evolve on its own. Statelessness requires that every request from a client to a server contain all the information necessary to understand and complete that request; the server holds no session state about the client between requests. Cacheability requires that responses define themselves as cacheable or non-cacheable so clients and intermediaries can reuse data safely. A layered system means a client cannot ordinarily tell whether it is connected directly to the origin server or to an intermediary such as a load balancer or gateway. A uniform

interface is the most distinctive constraint, standardizing how resources are named, manipulated, and represented. Code-on-demand is an optional constraint allowing servers to extend client functionality by transferring executable code.

In a RESTful design, everything of interest — a customer, an order, an invoice line item — is modeled as a resource, and each resource is addressed by a unique Uniform Resource Identifier (URI). Because statelessness is enforced, scalability follows almost automatically: any server instance behind a load balancer can service any request, since no server needs to remember prior interactions with a particular client. This same separation of concerns is what allows a frontend team (building a React interface, for example) and a backend team (building a Laravel API) to work independently against an agreed-upon contract.

Key Terms: **API** — a defined contract of operations and data formats that lets one software system consume another's functionality without exposing internal implementation. | **REST** — an architectural style for networked applications built around a small set of constraints — statelessness, a uniform interface, and cacheability among them. | **Resource** — any named piece of information (a document, image, or abstract concept like a customer record) that can be addressed and manipulated through the API. | **URI** — Uniform Resource Identifier; the address used to locate and act upon a specific resource. | **Statelessness** — the constraint that no client context is stored on the server between requests; each request is self-contained. | **Idempotency** — a property of an operation where performing it multiple times produces the same result as performing it once (e.g., PUT and DELETE are expected to be idempotent). | **HATEOAS** — Hypermedia as the Engine of Application State; a REST constraint where responses include links that guide the client to related or subsequent actions.

Example 1 — A Layered REST Architecture (Textual Diagram)

A typical production REST deployment separates concerns across several layers, none of which the client can distinguish from a single monolithic server.

```
[ React SPA Client ]
  | HTTPS (JSON over HTTP)
  v
[ Load Balancer / Reverse Proxy ]
  |
  v
[ Laravel API Server(s) ] <-- stateless: any instance can serve any request
  |
  v
[ Cache Layer (Redis) ] --- [ Database (MySQL/PostgreSQL) ]
```

Example 2 — Resource-Oriented URI Hierarchy

Well-designed REST URIs read as a hierarchy of nouns (resources), never verbs — the HTTP method supplies the verb.

```
GET /api/v1/customers/42          -> a single customer
GET /api/v1/customers/42/orders  -> that customer's orders (collection)
```

```
GET /api/v1/customers/42/orders/17 -> a specific order
```

```
# Avoid verbs in the URI:
# BAD: GET /api/getCustomerOrders?id=42
# GOOD: GET /api/v1/customers/42/orders
```

(b) HTTP Fundamentals

The Hypertext Transfer Protocol (HTTP) is the application-layer protocol that underlies virtually all RESTful APIs. Every interaction follows a request/response cycle: the client constructs a request consisting of a method, a target URI, a set of headers, and an optional body; the server processes that request and returns a response consisting of a status line, response headers, and an optional body. Understanding this cycle precisely — not just superficially — is essential, because the correctness of an API's design depends on using each part of the protocol for its intended purpose.

HTTP methods carry specific semantic meaning that clients and intermediaries (proxies, caches, browsers) rely on. GET retrieves a representation of a resource and must be safe (it must not change server state) and idempotent. POST creates a new resource or triggers a processing action, and is neither safe nor idempotent. PUT replaces a resource entirely and is idempotent — sending the same PUT twice produces the same end state. PATCH applies a partial modification and is not guaranteed to be idempotent unless designed carefully. DELETE removes a resource and is idempotent (deleting an already-deleted resource is still 'deleted'). Using the wrong method — for example, using GET to delete a record — breaks caching, breaks idempotency guarantees, and can lead to accidental data loss when a browser or crawler pre-fetches a GET request.

Status codes communicate the outcome of a request using a structured numeric taxonomy. The 1xx class is informational and rarely seen directly by application developers. The 2xx class indicates success (200 OK, 201 Created, 204 No Content). The 3xx class indicates redirection (301 Moved Permanently, 304 Not Modified). The 4xx class indicates a client error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 422 Unprocessable Entity). The 5xx class indicates a server error (500 Internal Server Error, 503 Service Unavailable). Choosing the precise, correct status code — rather than defaulting everything to 200 or 500 — is what allows API clients to build reliable, automated error-handling logic.

Key Terms: **HTTP Method** — a verb (GET, POST, PUT, PATCH, DELETE, etc.) that expresses the intended action to perform on a resource. | **Safe Method** — a method that does not alter server state (e.g., GET, HEAD). | **Idempotent Method** — a method where repeating the identical request multiple times has the same effect as sending it once (e.g., PUT, DELETE). | **Status Code** — a three-digit numeric code in the response's status line indicating the outcome class (informational, success, redirection, client error, server error). | **Header** — a key-value pair carried in the request or response that conveys metadata such as content type, authentication credentials, or caching rules. | **MIME Type** — a standardized label (e.g., application/json) describing the format of the request or response body, communicated via the Content-Type header.

Example 1 — A Raw HTTP Request

This is what actually travels over the wire when a client fetches a specific order resource, including the headers a REST client is expected to send.

```
GET /api/v1/orders/17 HTTP/1.1
Host: api.example.com
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
User-Agent: PostmanRuntime/7.36.0
```

Example 2 — The Corresponding HTTP Response

The server's response mirrors the request's structure: a status line, headers, then a body.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: no-store

{
  "id": 17,
  "customer_id": 42,
  "status": "shipped",
  "total": 1899.00
}
```

Example 3 — Method Semantics at a Glance

A quick reference for which methods are safe and/or idempotent — a common source of design mistakes.

Method	Safe?	Idempotent?	Typical Use
GET	Yes	Yes	Retrieve a resource or collection
POST	No	No	Create a new resource / trigger an action
PUT	No	Yes	Replace a resource entirely
PATCH	No	No*	Partially update a resource
DELETE	No	Yes	Remove a resource

(c) Building a Simple RESTful API

Moving from theory to a production-ready implementation requires deliberate strategies in three areas that are frequently overlooked by beginners: versioning, error handling, and authentication. API versioning acknowledges that an API's contract will change over time, and that existing client integrations must not break when it does. The two dominant strategies are URI versioning (embedding the version in the path, e.g., /api/v1/orders) and header versioning (specifying the version in an Accept or custom header, e.g., Accept: application/vnd.example.v2+json). URI versioning is simpler to discover and debug, while header versioning keeps URIs stable but requires more disciplined client tooling.

Error handling in a well-designed API is standardized rather than ad hoc. Instead of returning inconsistent, free-form error strings, mature APIs return a predictable JSON error envelope containing a machine-readable error code, a human-readable message, and optionally a list of field-level validation failures. This predictability is what allows a frontend to build generic error-handling logic once, rather than special-casing every endpoint. The IETF's RFC 7807 ("Problem Details for HTTP APIs") formalizes one widely adopted convention for this envelope.

Because REST APIs are stateless, authentication cannot rely on server-side sessions; instead, every request must carry proof of identity. The dominant approach for modern APIs is token-based authentication using JSON Web Tokens (JWTs) or opaque bearer tokens issued after a login step, sent on every subsequent request via the Authorization header. This keeps the server stateless (it can validate a JWT's signature without a database lookup) while still enforcing access control per request.

Key Terms: API Versioning — a deliberate strategy for evolving an API's contract over time without breaking existing client integrations. | **JWT (JSON Web Token)** — a compact, self-contained, digitally signed token that encodes claims (such as user identity and expiration) and can be verified without a server-side session store. | **Bearer Token** — a security token that grants access to whoever presents it ("bears" it) in the Authorization header, typically as Authorization: Bearer <token>. | **Problem Details (RFC 7807)** — a standardized JSON structure for representing API error responses, including fields such as type, title, status, and detail. | **Rate Limiting** — a control that restricts how many requests a client may make in a given time window, protecting the API from abuse or overload.

Example 1 — URI vs. Header Versioning

Two equally valid ways to signal which contract version a client expects.

```
# URI versioning
GET /api/v1/orders/17
GET /api/v2/orders/17

# Header versioning (URI stays stable)
GET /api/orders/17
Accept: application/vnd.example.v2+json
```

Example 2 — A Standardized JSON Error Payload

Rather than returning a bare string, a validation failure returns a structured, predictable envelope the frontend can parse generically.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "The given data was invalid.",
    "fields": {
```

```

    "email": ["The email field is required."],
    "price": ["The price must be a positive number."]
  }
}
}

```

Example 3 — Anatomy of a JWT

A JWT is three Base64URL-encoded segments — header, payload, and signature — separated by periods. Only the signature (verified with a server secret) needs cryptographic checking; the rest is simply decoded.

```

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.
eyJzdWIiOiI0MiIsIm5hbWUiOiJKdWFiIiwiaXNjaXNzMDAwMDAwfQ
.
4f8f1c2e9b6a7d... (HMAC-SHA256 signature)

# Decoded payload (claims):
{ "sub": "42", "name": "Juan", "exp": 1730000000 }

```

Laboratory Activities

Lab 1: Implement CRUD Endpoints Mapped to HTTP Verbs and Resource-Oriented URIs

Prerequisites & Tools:

- PHP 8.2+ and Composer installed (see the midterm handout's Laravel installation guide if not yet set up).
- A working Laravel project (composer create-project laravel/laravel api-project).
- A REST client for testing: Postman, Insomnia, or cURL.
- A configured database connection (SQLite is sufficient for this lab).

Step-by-Step Instructions:

1. Generate a model and migration for the resource this lab will expose:

```
php artisan make:model Task -m
```

2. Define the table schema in the generated migration file, then run it:

```

// database/migrations/xxxx_create_tasks_table.php
Schema::create('tasks', function (Blueprint $table) {
    $table->id();
    $table->string('title');
    $table->boolean('is_done')->default(false);

```

```
$table->timestamps();
});
```

```
php artisan migrate
```

3. Generate an API resource controller:

```
php artisan make:controller Api/TaskController --api
```

4. Register resource-oriented routes in routes/api.php, mapping each HTTP verb to the correct action:

```
// routes/api.php
use App\Http\Controllers\Api\TaskController;

Route::apiResource('tasks', TaskController::class);
// Generates:
// GET    /api/tasks          -> index
// POST   /api/tasks          -> store
// GET    /api/tasks/{task}  -> show
// PUT    /api/tasks/{task}  -> update
// DELETE /api/tasks/{task}  -> destroy
```

5. Implement each controller method using Eloquent so every verb performs the correct CRUD operation:

```
// app/Http/Controllers/Api/TaskController.php
public function index() { return Task::latest()->get(); }
public function store(Request $r) { return Task::create($r->validate(['title'=>'required|string'])); }
public function show(Task $task) { return $task; }
public function update(Request $r, Task $task) {
    $task->update($r->validate(['title'=>'sometimes|string', 'is_done'=>'sometimes|boolean']));
    return $task;
}
public function destroy(Task $task) { $task->delete(); return response()->noContent(); }
}
```

6. Test every endpoint in Postman: create a task with POST, list tasks with GET, update one with PUT, and remove it with DELETE, confirming each returns the expected status code (201, 200, 200, 204 respectively).

Expected Output:

```
POST /api/tasks { "title": "Write lab report" }
-> 201 Created
{ "id": 1, "title": "Write lab report", "is_done": false, ... }
```

```
GET /api/tasks
-> 200 OK
[ { "id": 1, "title": "Write lab report", "is_done": false, ... } ]

DELETE /api/tasks/1
-> 204 No Content
```

Lab 2: Develop Validation Middleware and Structured Error Responses

Prerequisites & Tools:

- The Laravel project and Task API from Lab 1.
- Postman or cURL for triggering invalid requests.

Step-by-Step Instructions:

7. Generate a dedicated Form Request class instead of validating inline, to keep controllers thin:

```
php artisan make:request StoreTaskRequest
```

8. Define the validation rules and authorization check:

```
// app/Http/Requests/StoreTaskRequest.php
public function authorize(): bool { return true; }
public function rules(): array {
    return [
        'title' => 'required|string|max:255',
        'is_done' => 'sometimes|boolean',
    ];
}
```

9. Update the controller to type-hint the Form Request so validation runs automatically before the method body executes:

```
public function store(StoreTaskRequest $request)
{
    return Task::create($request->validated());
}
```

10. Customize the JSON error envelope globally so every validation failure (and not just this endpoint) returns a consistent structure, by overriding the exception handler's render logic in bootstrap/app.php (Laravel 11) or app/Exceptions/Handler.php (Laravel 10):

```
use Illuminate\Validation\ValidationException;

->withExceptions(function ($exceptions) {
```

```

    $exceptions->render(function (ValidationException $e, $request) {
        if ($request->is('api/*')) {
            return response()->json([
                'error' => [
                    'code' => 'VALIDATION_FAILED',
                    'message' => 'The given data was invalid.',
                    'fields' => $e->errors(),
                ],
            ], 422);
        }
    });
})

```

11. Send a POST request with an empty title field and confirm the API returns HTTP 422 with the standardized error envelope, not Laravel's default HTML error page.

Expected Output:

```

POST /api/tasks { "title": "" }
-> 422 Unprocessable Entity
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "The given data was invalid.",
    "fields": { "title": ["The title field is required."] }
  }
}

```

Lab 3: Construct a Persistence Layer That Reliably Commits State Changes

Prerequisites & Tools:

- The Laravel project, Task model, and migrations from Labs 1-2.
- php artisan tinker or a database viewer (e.g., TablePlus, DBeaver) to inspect table contents directly.

Step-by-Step Instructions:

12. Confirm the tasks table exists and is empty using php artisan tinker (Task::count() should return 0).
13. Issue a POST request through the API, then immediately re-query the database directly via Tinker to confirm the row was actually persisted (not just returned in the response):

```

php artisan tinker
>>> Task::latest()->first()

```

14. Wrap a multi-step operation — for example, creating a task and logging an audit record — in a database transaction, so that either both writes succeed or neither does:

```

use Illuminate\Support\Facades\DB;

```

```
DB::transaction(function () use ($request) {
    $task = Task::create($request->validated());
    AuditLog::create(['action' => 'task_created', 'task_id' => $task->id]);
});
```

15. Deliberately force the second write to fail (e.g., by violating a NOT NULL constraint on AuditLog) and confirm, via Tinker, that the transaction rolled back and the Task record was NOT left behind — proving the persistence layer maintains data integrity even under partial failure.
16. Issue PUT and DELETE requests and confirm, via direct database inspection, that updates and deletions are reflected in the tasks table exactly as the API responses claimed.

Expected Output:

```
>>> Task::count();
=> 0
>>> // after POST /api/tasks
>>> Task::count();
=> 1
>>> // after forcing the audit-log write to fail inside DB::transaction
>>> Task::count();
=> 1 // unchanged -- confirms the transaction rolled back cleanly
```

Module 2: Cloud Computing and Deployment (Week 15)

Mapped CILOs (as given): CILO 3 (as given in the outline)

Topics: (a) Introduction to Cloud Computing Platforms, (b) Serverless Architecture and Microservices, (c) Deployment Strategies for Scalable Applications

CILO-to-ILO Alignment Table

#	Intended Learning Outcome (ILO)	CILO(s) Supported
Lecture ILOs		
1	Explain the key concepts, types, and benefits of cloud computing platforms.	CILO 1
2	Describe serverless computing and microservices design principles.	CILO 1
3	Explain strategies for deploying applications to ensure scalability and	CILO 1, CILO 2
Laboratory ILOs		
1	Access and navigate a cloud platform to deploy and manage simple resources.	CILO 1
2	Implement a basic serverless function or microservice on a cloud platform.	CILO 1, CILO 2
3	Deploy a sample application using cloud services with scalable configurations.	CILO 1, CILO 2

(a) Introduction to Cloud Computing Platforms

Cloud computing is the on-demand delivery of computing resources — servers, storage, databases, networking, and software — over the internet, billed on a pay-as-you-go basis rather than requiring an organization to purchase and maintain physical infrastructure. This model fundamentally shifts capital expenditure (buying hardware upfront) into operational expenditure (paying for what is actually consumed), and it is precisely this shift, combined with elastic scalability, that has made cloud platforms the default deployment target for modern applications.

Cloud services are conventionally organized into three delivery models. Infrastructure as a Service (IaaS) provides raw virtualized compute, storage, and networking — Amazon EC2, Azure Virtual Machines, and Google Compute Engine are examples — leaving the customer responsible for the operating system and everything above it. Platform as a Service (PaaS) goes further, providing a managed runtime, database, and middleware so developers can deploy application code directly without managing servers at all — Heroku, Google App Engine, and Azure App Service are examples. Software as a Service (SaaS) delivers a complete, ready-to-use application over the internet, such as Google Workspace or Salesforce, where the customer manages neither infrastructure nor the application code itself.

Cloud platforms are also classified by ownership and tenancy model. A public cloud (AWS, Azure, GCP) is owned and operated by a third-party provider and shared across many customers, offering the greatest scalability and the lowest barrier to entry. A private cloud is dedicated to a single organization, either hosted on-premises or by a third party, trading some scalability and cost-efficiency for greater control and security guarantees. A hybrid cloud combines both, allowing sensitive workloads to remain on a private cloud while burstable or public-facing workloads run on a public cloud. Choosing among these

models requires weighing cost, scalability, compliance requirements, and the organization's tolerance for vendor lock-in.

Key Terms: **IaaS** — Infrastructure as a Service; provides virtualized compute, storage, and networking that the customer configures and manages above the hypervisor layer. | **PaaS** — Platform as a Service; provides a managed application runtime and supporting services so developers deploy code without managing servers. | **SaaS** — Software as a Service; delivers a fully built application to end users over the internet on a subscription basis. | **Public Cloud** — cloud infrastructure owned by a third-party provider and shared across multiple customers over the public internet. | **Private Cloud** — cloud infrastructure dedicated to a single organization, offering greater control at the cost of some scalability and cost-efficiency. | **Elasticity** — the ability of a cloud platform to automatically provision or de-provision resources in response to real-time demand. | **Vendor Lock-In** — the risk and cost of dependency on a single cloud provider's proprietary services, making migration to another provider difficult.

Example 1 — The IaaS / PaaS / SaaS Responsibility Split

As you move from IaaS to SaaS, the cloud provider takes on progressively more of the operational responsibility, and the customer retains progressively less control.

Layer	On-Premises	IaaS	PaaS	SaaS
Application code	You	You	You	Provider
Runtime	You	You	Provider	Provider
Operating System	You	You	Provider	Provider
Virtualization	You	Provider	Provider	Provider
Servers/Storage/Net	You	Provider	Provider	Provider

Example 2 — A Real-World Platform Selection Scenario

A startup building a REST API (from Module 1) needs to choose a hosting model. If they deploy the Laravel API on a raw virtual machine they manage themselves, that is IaaS (e.g., an AWS EC2 instance). If they instead push the same code to a managed platform like AWS Elastic Beanstalk or Heroku and let the platform handle scaling and patching, that is PaaS. Neither choice is 'more correct' — the decision depends on how much operational control the team needs versus how much time they want to spend on infrastructure.

(b) Serverless Architecture and Microservices

Serverless computing, more precisely called Function as a Service (FaaS), abstracts server management away entirely. Developers write small, single-purpose functions that are triggered by events — an HTTP request, a file upload, a message arriving on a queue — and the cloud provider handles provisioning, scaling, and tearing down the underlying compute automatically. Billing is per-invocation and per-resource-consumed, rather than for a continuously running server, which makes serverless extremely cost-efficient for workloads with unpredictable or spiky traffic. The trade-offs include cold-start latency (a brief delay the first time an idle function is invoked), tighter execution-time limits, and a degree of vendor lock-in to a provider's specific event and deployment model.

Microservices architecture takes a different but complementary angle: instead of building one large, monolithic application, the system is decomposed into small, independently deployable services, each responsible for a single business capability (an Orders service, a Payments service, an Inventory service). Each service can be built, tested, deployed, scaled, and even rewritten in a different technology stack independently of the others, which accelerates release cycles and improves fault isolation — a failure in the Payments service does not necessarily bring down the entire application. The cost of this independence is increased operational complexity: services must communicate over a network (introducing latency and partial-failure modes that don't exist inside a single process), and maintaining data consistency across services requires deliberate patterns such as eventual consistency or the saga pattern.

Serverless functions and microservices are frequently combined rather than treated as competing choices. A microservices-based system might implement one particular capability — say, resizing an uploaded image — as a serverless function, because that workload is bursty and event-driven, while implementing its core Orders service as a persistently running container. Understanding when to reach for each pattern is itself a design skill: serverless suits short-lived, event-triggered, spiky workloads, while long-running, stateful, or latency-sensitive services are usually better served by a conventionally deployed microservice.

Key Terms: Serverless / FaaS — a cloud execution model where developers deploy individual functions that run in response to events, with the provider fully managing the underlying servers. | **Cold Start** — the latency incurred the first time a serverless function is invoked after being idle, while the provider provisions an execution environment for it. | **Microservices** — an architectural style that decomposes an application into small, independently deployable services, each owning a specific business capability. | **Event-Driven** — a design pattern where code executes in response to an event (an HTTP call, a queue message, a file upload) rather than running continuously. | **Saga Pattern** — a pattern for maintaining data consistency across multiple microservices by coordinating a sequence of local transactions, each with a corresponding compensating action.

Example 1 — A Minimal AWS Lambda Function (Node.js)

A serverless function that responds to an HTTP event via API Gateway, demonstrating the event-driven, stateless execution model.

```
// handler.js
exports.handler = async (event) => {
  const name = event.queryStringParameters?.name || 'World';
  return {
    statusCode: 200,
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ message: `Hello, ${name}!` }),
  };
};
```

Example 2 — Monolith vs. Microservices (Textual Diagram)

Decomposing a single Laravel monolith into cooperating services, each independently deployable and scalable.

MONOLITH:	MICROSERVICES:
[Single Laravel App]	[Orders Service] ---\
- Orders	\
- Payments	[Payments Service] --- (message queue / API calls)
- Inventory	/
- Single Database	[Inventory Service] --/
	(each with its own database)

(c) Deployment Strategies for Scalable Applications

Once an application is built, how it is deployed and scaled directly determines whether it can handle real-world load without downtime or degraded performance. Horizontal scaling ("scaling out") adds more instances of an application behind a load balancer, distributing incoming traffic across them; it is the preferred strategy for stateless applications (a natural fit for the stateless REST APIs from Module 1) because any instance can serve any request. Vertical scaling ("scaling up") instead increases the resources — CPU, memory — of existing instances; it is simpler to manage since the instance count stays fixed, but it has a hard ceiling (a single machine can only be upgraded so far) and typically requires downtime to resize.

Containerization packages an application together with its dependencies into a lightweight, portable unit (a container) that runs consistently across development, staging, and production environments, eliminating the classic 'it works on my machine' failure mode. Container orchestration platforms, most notably Kubernetes, then automate the deployment, scaling, health-checking, and self-healing of many containers across a cluster of machines, restarting failed containers automatically and redistributing load as instances come and go.

Deployment strategy also governs how a new version of an application is rolled out to production without causing downtime or excessive risk. In a blue-green deployment, two identical production environments ("blue" and "green") exist simultaneously; traffic is switched from the currently active environment to the newly deployed one instantly, and a rollback is simply switching traffic back. In a canary deployment, the new version is rolled out gradually to a small subset of real traffic first, while the team monitors error rates and performance, before progressively increasing the percentage of traffic routed to the new version. Both strategies exist to minimize the blast radius of a bad deployment.

Key Terms: **Horizontal Scaling** — increasing capacity by adding more instances of an application behind a load balancer ("scaling out"). | **Vertical Scaling** — increasing capacity by adding more CPU or memory to an existing instance ("scaling up"). | **Load Balancer** — a component that distributes incoming network traffic across multiple backend instances. | **Containerization** — packaging an application and its dependencies into a portable, consistent runtime unit (e.g., a Docker container). | **Orchestration** — the automated management of container deployment, scaling, networking, and health

across a cluster (e.g., Kubernetes). | **Blue-Green Deployment** — a zero-downtime release strategy that maintains two production environments and switches traffic between them instantly. | **Canary Deployment** — a gradual release strategy that routes a small percentage of traffic to a new version before a full rollout.

Example 1 — Horizontal Scaling Behind a Load Balancer (Textual Diagram)

Because the REST API from Module 1 is stateless, any of the instances below can correctly serve any incoming request.



Example 2 — A Minimal Dockerfile for the Laravel API

Containerizing the API from Module 1 so it runs identically across every environment.

```

FROM php:8.2-fpm
WORKDIR /var/www
COPY . .
RUN apt-get update && apt-get install -y unzip \
    && curl -sS https://getcomposer.org/installer | php \
    && php composer.phar install --no-dev --optimize-autoloader
EXPOSE 9000
CMD ["php-fpm"]
  
```

Example 3 — Canary Rollout Traffic Split

A canary deployment gradually increases exposure to a new version while monitoring for errors, rather than switching 100% of traffic at once.

Stage 1:	v1 = 95% of traffic	v2 (canary) = 5%	
Stage 2:	v1 = 75%	v2 = 25%	<- proceed only if error rate is normal
Stage 3:	v1 = 0%	v2 = 100%	<- full rollout complete

Laboratory Activities

Lab 1: Access and Navigate a Cloud Platform to Deploy and Manage Simple Resources

Prerequisites & Tools:

- A free-tier account on a public cloud provider (AWS, Microsoft Azure, or Google Cloud Platform — this lab uses AWS terminology, but the steps map directly to any provider's console).
- A modern web browser to access the cloud provider's management console.

- (Optional) The provider's CLI tool (e.g., AWS CLI) installed for command-line verification.

Step-by-Step Instructions:

17. Sign in to the AWS Management Console and locate the EC2 (Elastic Compute Cloud) service.
18. Launch a new virtual machine instance: choose a free-tier-eligible Amazon Machine Image (AMI) such as Amazon Linux 2023, select the t2.micro instance type, and generate or select an SSH key pair.
19. Configure a security group that allows inbound SSH (port 22) from your IP and HTTP (port 80) from anywhere, then launch the instance.
20. Once the instance is running, connect to it via SSH using the key pair you generated:

```
ssh -i my-key.pem ec2-user@<public-ip-address>
```

21. Install a simple web server on the instance (e.g., `sudo yum install -y httpd && sudo systemctl start httpd`) and confirm you can reach the default page from your browser using the instance's public IP address.
22. Navigate to the S3 (storage) service in the same console and create a simple storage bucket, uploading a test file to confirm you can manage a second resource type from the same platform.
23. Terminate the EC2 instance and delete the S3 bucket once the exercise is complete, to avoid incurring unexpected charges.

Expected Output:

The EC2 instance reaches a 'running' state in the console, its public IP serves the default Apache test page in a browser, and the uploaded file appears in the S3 bucket's object list — confirming successful navigation and management of two distinct cloud resource types.

Lab 2: Implement a Basic Serverless Function on a Cloud Platform

Prerequisites & Tools:

- The same cloud provider account from Lab 1.
- Node.js installed locally (for writing and testing the function logic before upload).
- The AWS CLI configured with valid credentials (`aws configure`), if deploying from the command line.

Step-by-Step Instructions:

24. Write a minimal Lambda handler function locally that responds to an HTTP event:

```
// handler.js
exports.handler = async (event) => {
  return {
    statusCode: 200,
    body: JSON.stringify({ message: 'Hello from a serverless function!' }),
  };
};
```

25. Package the function into a ZIP archive:

```
zip function.zip handler.js
```

26. Create the Lambda function via the console or CLI:

```
aws lambda create-function \
  --function-name hello-serverless \
  --runtime nodejs18.x \
  --handler handler.handler \
  --zip-file fileb://function.zip \
  --role arn:aws:iam::<account-id>:role/lambda-basic-execution
```

27. In the AWS Console, add an API Gateway HTTP API trigger to the function so it becomes reachable over the internet via a public URL.

28. Invoke the deployed function and confirm it returns the expected JSON payload:

```
curl https://<api-id>.execute-api.<region>.amazonaws.com/hello-serverless
```

Expected Output:

```
$ curl https://abc123.execute-api.us-east-1.amazonaws.com/hello-serverless
{"message":"Hello from a serverless function!"}
```

Lab 3: Deploy a Sample Application Using Cloud Services with Scalable Configurations

Prerequisites & Tools:

- The containerized Laravel API from the Module 2 lecture example (or the CRUD API from Module 1, Lab 1).
- Docker installed locally to build and test the container image.
- An account with a PaaS or container-hosting service (e.g., AWS Elastic Beanstalk, Google Cloud Run, or Heroku).

Step-by-Step Instructions:

29. Build and locally test the Docker image for the API:

```
docker build -t my-rest-api .
docker run -p 8000:9000 my-rest-api
```

30. Push the built image to a container registry (e.g., Amazon ECR or Docker Hub) so the cloud platform can pull it during deployment.

31. Create a new environment on the chosen PaaS (e.g., an Elastic Beanstalk environment or a Cloud Run service) and point it at the pushed container image.
32. Configure an auto-scaling policy so the platform adds instances automatically under load — for example, scaling out when average CPU utilization exceeds 70%:

```
# Elastic Beanstalk scaling configuration (excerpt)
MinSize: 1
MaxSize: 4
ScalingTrigger: CPUUtilization > 70%
```

33. Use a load-testing tool (e.g., Apache Bench: `ab -n 2000 -c 50 https://<deployed-url>/api/tasks`) to generate sustained traffic and observe, in the platform's monitoring dashboard, that additional instances are provisioned automatically as load increases and removed again once load subsides.

Expected Output:

The monitoring dashboard shows the instance count rising from 1 to several during the load test and returning to 1 afterward, while the API continues to return 200 OK responses throughout — confirming the scalable configuration correctly maintains availability under variable load.

Module 3: Emerging Technologies: Artificial Intelligence (Weeks 16-17)

Mapped CILOs (as given): CILO 3 (as given in the outline)

Topics: (a) Introduction to AI and Machine Learning, (b) AI in Application Development, (c) Ethical Considerations in AI Application Development

CILO-to-ILO Alignment Table

#	Intended Learning Outcome (ILO)	CILO(s) Supported
Lecture ILOs		
1	Explain the fundamentals of AI and machine learning, including types, concepts, and applications.	CILO 1
2	Describe how AI techniques are integrated into software applications to solve real-world problems.	CILO 1
3	Explain key ethical issues in AI, including bias, fairness, privacy, and accountability.	CILO 2
Laboratory ILOs		
1	Apply basic machine learning algorithms on sample datasets using a programming environment.	CILO 1
2	Implement a simple AI feature (e.g., classification, recommendation, or prediction) in a sample application.	CILO 1, CILO 2
3	Analyze sample AI models or datasets to identify potential ethical concerns and suggest mitigation strategies.	CILO 2, CILO 3

(a) Introduction to AI and Machine Learning

Artificial Intelligence (AI) is the branch of computer science concerned with building systems that simulate aspects of human intelligence — learning, reasoning, perception, language understanding, and decision-making. AI is commonly divided into Narrow AI (also called Weak AI), which is designed and trained for a specific task and cannot generalize beyond it (a spam filter, a voice assistant, a recommendation engine), and General AI (Strong AI), a hypothetical system possessing broad, human-like cognitive ability across arbitrary domains. Every AI application in production today — including everything covered in this module — is Narrow AI; General AI remains a research aspiration rather than an engineering reality.

Machine Learning (ML) is the dominant technique used to build modern Narrow AI systems. Rather than being explicitly programmed with rules, an ML model learns patterns directly from data. Supervised learning trains a model on labeled examples (input-output pairs) so it can predict the output for new, unseen inputs — for example, training a model on historical house sales (inputs: square footage, location, bedrooms; output: sale price) so it can estimate the price of a new listing. Unsupervised learning finds structure in unlabeled data, such as clustering customers into behavioral segments without being told in advance what those segments are. Reinforcement learning trains an agent to make sequential decisions by rewarding or penalizing outcomes, as used in game-playing AI and robotics.

Within supervised learning, two broad problem types dominate application development. Classification predicts a discrete category (is this email spam or not spam?), while regression predicts a continuous numeric value (what will this house sell for?). Understanding which problem type a business requirement maps to is the first and most important step in designing any AI feature, because it determines which algorithms, evaluation metrics, and data preparation steps are appropriate.

Key Terms: Artificial Intelligence (AI) — the field of computer science focused on building systems that simulate aspects of human intelligence. | **Narrow AI (Weak AI)** — AI designed and trained to perform a specific task, without the ability to generalize beyond it — the only form of AI that exists in production today. | **General AI (Strong AI)** — a hypothetical AI possessing broad, human-like cognitive ability across arbitrary domains; not yet realized in practice. | **Machine Learning (ML)** — a subset of AI in which a model learns patterns directly from data rather than being explicitly programmed with rules. | **Supervised Learning** — an ML approach that trains a model on labeled input-output pairs so it can predict outputs for new inputs. | **Unsupervised Learning** — an ML approach that discovers structure (such as clusters) in unlabeled data. | **Classification** — a supervised learning task that predicts a discrete category or label. | **Regression** — a supervised learning task that predicts a continuous numeric value.

Example 1 — Classification vs. Regression

The same dataset can support either problem type depending on what question is being asked.

```
Dataset: house sales (sqft, location, bedrooms, sale_price)

Regression:      predict the exact sale_price (a continuous number)
Classification:  predict whether sale_price > $500,000 (Yes / No)
```

Example 2 — A Minimal Supervised Learning Workflow (Python / scikit-learn)

The canonical shape of nearly every supervised ML project: split data, train, predict, evaluate.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LogisticRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, predictions))
```

(b) AI in Application Development

Integrating AI into a software application rarely means training a model from scratch inside the application itself. In practice, developers most often consume AI capability in one of three ways: calling a

pre-trained cloud AI API (e.g., a sentiment-analysis or image-recognition endpoint) over HTTPS exactly like any other REST API from Module 1; embedding a pre-trained open-source model (e.g., via a Python library) directly into the application's backend; or training a lightweight custom model on the organization's own data for a narrow, well-defined task such as churn prediction.

The most common application patterns are classification (routing support tickets to the correct department), recommendation (suggesting products or content based on user behavior), and prediction/forecasting (estimating future demand or risk). Regardless of which pattern is used, the AI component is typically wrapped behind an ordinary API endpoint, so the rest of the application — the React frontend, the Laravel controllers — interacts with it exactly as it would with any other data source, requesting a prediction and receiving a JSON response.

A critical, often underestimated part of integrating AI is monitoring model performance in production. Unlike traditional software, whose behavior is deterministic once shipped, an ML model's accuracy can silently degrade over time as real-world data drifts away from the distribution it was trained on — a phenomenon called model drift. Production AI features therefore need the same operational discipline as any other emerging technology deployed to the cloud (Module 2): logging predictions, tracking accuracy against ground truth as it becomes available, and having a plan to retrain or roll back a model whose performance has degraded.

Key Terms: Pre-trained Model — a model already trained on a large, general dataset that can be used directly or fine-tuned for a specific application. | **Inference** — the process of using a trained model to generate a prediction for new, unseen input. | **Recommendation System** — an AI application that predicts and suggests items (products, content) a user is likely to want, based on patterns in behavioral data. | **Model Drift** — the degradation of a deployed model's accuracy over time as real-world data diverges from its training distribution. | **MLOps** — the set of practices for deploying, monitoring, and maintaining machine learning models in production, analogous to DevOps for traditional software.

Example 1 — Consuming a Cloud AI API from a Laravel Backend

AI capability is frequently consumed exactly like any REST API from Module 1 — the calling code does not need to know a model is involved at all.

```
// app/Http/Controllers/Api/SentimentController.php
public function analyze(Request $request)
{
    $response = Http::withToken(config('services.ai.key'))
        ->post('https://api.ai-provider.example/v1/sentiment', [
            'text' => $request->input('text'),
        ]);

    return response()->json($response->json());
}
```

Example 2 — A Sample Prediction Response

Whatever pattern is used (classification, recommendation, or prediction), the result is typically returned as a plain JSON payload, following the same standardized conventions established in Module 1.

```
POST /api/sentiment { "text": "The checkout process was confusing." }
-> 200 OK
{
  "label": "negative",
  "confidence": 0.87
}
```

(c) Ethical Considerations in AI Application Development

Because ML models learn patterns from historical data, they can also learn and amplify the biases present in that data. Bias in AI refers to systematic, unfair skew in a model's predictions toward or against particular groups — for example, a hiring-recommendation model trained on a company's past (historically male-dominated) hiring decisions may learn to downrank female candidates, not because gender is an explicit input, but because it correlates with other features in the training data. Fairness in AI is the active discipline of measuring and correcting for this kind of skew, using techniques such as auditing model outputs across demographic subgroups and re-balancing or re-weighting training data.

Privacy is a second major ethical concern: many AI applications are trained on or process personal data (health records, browsing behavior, biometric data), and developers have both an ethical and, in many jurisdictions, a legal obligation (e.g., under GDPR or the Philippines' Data Privacy Act of 2012) to minimize data collection, anonymize or pseudonymize data where possible, and obtain informed consent. This directly connects to CILO 2's requirement to apply coding and design standards 'with respect to legal and ethical considerations' — privacy-by-design is not a separate add-on step but a core design constraint from the start of a project.

Accountability addresses the question of who is responsible when an AI system causes harm — a denied loan application, a misdiagnosed medical image, a wrongful arrest from facial recognition. Because ML models can behave as 'black boxes' whose internal reasoning is difficult to inspect, responsible AI development emphasizes explainability (being able to articulate why a model produced a particular output), maintaining a human-in-the-loop for consequential decisions, and thorough documentation of a model's training data, intended use cases, and known limitations — often published as a 'model card.'

Key Terms: **Bias (in AI)** — a systematic skew in a model's predictions that unfairly favors or disfavors particular groups, typically inherited from patterns in the training data. | **Fairness** — the practice of measuring, and actively correcting for, unequal model performance or outcomes across demographic subgroups. | **Privacy-by-Design** — an approach that builds data minimization, anonymization, and consent into a system's architecture from the outset, rather than retrofitting it later. | **Explainability** — the degree to which a model's decision-making process can be understood and articulated by humans. | **Accountability** — the assignment of responsibility for the outcomes and impacts of an AI system's decisions. | **Model Card** — a short, standardized document describing a model's intended use, training data, performance characteristics, and known limitations.

Example 1 — A Real-World Bias Scenario

A concrete illustration of how bias enters a model indirectly, even when a protected attribute is deliberately excluded from the input features.

Training data: 10 years of hiring decisions (90% of hires were male)
Features used: years_experience, school_tier, previous_job_title, zip_code

Problem: zip_code and school_tier correlate strongly with gender and socioeconomic background in the historical data, so the model learns a 'proxy' for gender even though gender itself was never an input.

Example 2 — Excerpt from a Model Card

Publishing this kind of documentation alongside a deployed model directly supports the accountability and explainability principles discussed above.

Model: churn-predictor-v3
Intended use: Flag at-risk subscription customers for retention outreach.
Training data: 18 months of anonymized subscription and usage logs.
Known limitations: Under-tested on customers with < 3 months of history; accuracy drops ~12% for that segment.
Last evaluated for demographic fairness: 2026-05-01

Laboratory Activities

Lab 1: Apply Basic Machine Learning Algorithms on Sample Datasets

Prerequisites & Tools:

- Python 3.10+ installed.
- The scikit-learn, pandas, and numpy libraries (pip install scikit-learn pandas numpy).
- A sample tabular dataset (e.g., the classic Iris dataset, bundled with scikit-learn).
- Jupyter Notebook or any Python-capable code editor.

Step-by-Step Instructions:

34. Load a sample dataset and inspect its structure:

```
from sklearn.datasets import load_iris
import pandas as pd

data = load_iris(as_frame=True)
df = data.frame
print(df.head())
print(df['target'].value_counts())
```

35. Split the dataset into training and testing sets to allow honest evaluation of the model afterward:

```
from sklearn.model_selection import train_test_split
X = df.drop(columns=['target'])
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)
```

36. Train a simple classification algorithm on the training set:

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)
```

37. Generate predictions on the held-out test set and evaluate accuracy:

```
from sklearn.metrics import accuracy_score, classification_report
predictions = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions))
```

38. Experiment with a second algorithm (e.g., KNeighborsClassifier) on the same split and compare accuracy scores, documenting which algorithm performed better and offering a brief hypothesis as to why.

Expected Output:

```
Accuracy: 0.9736842105263158
      precision    recall  f1-score   support
0         1.00      1.00      1.00        13
1         0.94      1.00      0.97        16
2         1.00      0.89      0.94          9
```

Lab 2: Implement a Simple AI Feature in a Sample Application

Prerequisites & Tools:

- The Laravel REST API from Module 1 (or a similar backend project).
- Python 3.10+ with scikit-learn and joblib (for saving/loading the trained model), or an equivalent Node.js ML library if the team prefers a single-language stack.
- A small labeled dataset relevant to the feature being built (e.g., historical support tickets labeled by category, for a classification feature).

Step-by-Step Instructions:

39. Train and export a simple classifier that will power the AI feature, e.g., a support-ticket category classifier:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

```

from sklearn.pipeline import Pipeline
import joblib

pipeline = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('clf', LogisticRegression(max_iter=1000)),
])
pipeline.fit(ticket_texts, ticket_categories)
joblib.dump(pipeline, 'ticket_classifier.joblib')

```

40. Wrap the trained model in a minimal Python inference API (e.g., using Flask or FastAPI) so other services can call it over HTTP, following the same REST conventions from Module 1:

```

# predict_service.py (FastAPI)
from fastapi import FastAPI
import joblib

app = FastAPI()
model = joblib.load('ticket_classifier.joblib')

@app.post('/predict')
def predict(payload: dict):
    category = model.predict([payload['text']])[0]
    return {'category': category}

```

41. From the Laravel application, call this inference endpoint exactly like any external REST API, using Laravel's HTTP client:

```

// app/Http/Controllers/Api/TicketController.php
$prediction = Http::post('http://localhost:8001/predict', [
    'text' => $request->input('description'),
])->json();

$ticket->update(['suggested_category' => $prediction['category']]);

```

42. Submit several new, unseen support-ticket descriptions through the Laravel endpoint and confirm each one receives a plausible category suggestion.

Expected Output:

```

POST /api/tickets { "description": "My payment was charged twice" }
-> 200 OK
{
  "id": 88,
  "description": "My payment was charged twice",
  "suggested_category": "billing"
}

```

Lab 3: Analyze Sample AI Models or Datasets for Ethical Concerns

Prerequisites & Tools:

- The trained classifier and dataset from Lab 1 or Lab 2.
- pandas for data analysis; matplotlib or seaborn for optional visualization.
- A worksheet or shared document for the team to record findings (this lab is best run as a small-group activity).

Step-by-Step Instructions:

43. As a team, profile the training dataset for representation imbalance across any available demographic or categorical attributes:

```
print(df.groupby('category')['outcome'].value_counts(normalize=True))
```

44. Evaluate the trained model's performance separately for each subgroup, rather than relying on a single overall accuracy figure, to check for disparate performance:

```
for group in df['segment'].unique():
    subset = df[df['segment'] == group]
    preds = model.predict(subset[features])
    print(group, accuracy_score(subset['target'], preds))
```

45. Discuss as a team which features in the dataset might act as proxies for a sensitive attribute (as in the zip_code/school_tier example from the lecture), even if that attribute was never used directly as an input.
46. Document at least two concrete ethical concerns identified (e.g., under-representation of a subgroup, disparate accuracy across segments, a plausible proxy variable) and propose a specific mitigation for each (e.g., collecting more balanced data, re-weighting training examples, removing or transforming a proxy feature, adding a human review step before any consequential decision is finalized).
47. Summarize the findings in a short model card (see the lecture example) and submit it as the team's deliverable for this lab.

Expected Output:

A completed team worksheet or model card documenting at least two specific ethical findings (e.g., 'Segment B is under-represented at 8% of training data and shows 15% lower accuracy') along with a corresponding, actionable mitigation strategy for each finding.

